University No. | | | | | - | | | - | | | | | Seat No. __________

SCHOOL OF COMPUTING & DATA SCIENCE
The University of Hong Kong

COMP2113 Programming Technologies

Sample Examination Paper (Long Questions)

(Suggested Solutions)

<u>Instructions</u>

Answer ALL questions.

Students are permitted to bring to the examination one sheet of A4-sized paper with printed or written notes on both sides.

Only approved calculators as announced by the Examinations Secretary can be used in this examination. It is the candidates' responsibility to ensure that their calculator operates satisfactorily, and candidates must record the name and type of the calculator on the first page of the answer script.

Question 1. Shell script

Please make sure that your written scripts are complete so that they can actually run. Please make you script as short and clean as possible, because some scripts need only one command line.

(a) Write a script which will print numbers from 0 to 100 and display every third (0 3 6 9 …).

Answer:

```
#!/bin/bash
for i in {0..100..3}; do echo $i; done
or
for (( i=0; i<=100; i=i+3 )); do echo $i; done
```

(b) Create an empty, read-only file called "foo.txt" in current directory.

Answer:

```
#!/bin/bash
touch foo.txt
chmod 400 foo.txt

(Note: rwx = 111 = 7, rw- = 110 = 6, r-- = 100 = 4, etc.)
```

Question 2. Makefile

Suppose we have the following collection of C header and implementation files:

<pre>-------------------- 	
thing.h
-------------------- 	
#ifndef THING_H
#define THING_H
#include "gadget.h"
...
#endif</pre>
	<pre>-------------------- 	
	thing.c
	-------------------- 	
	#include "thing.h"
	...</pre>

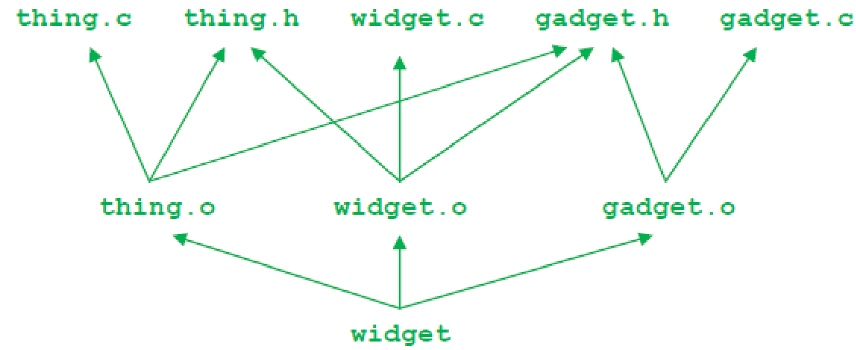
<pre>-------------------- 	
gadget.h
-------------------- 	
#ifndef GADGET_H
#define GADGET _H
...
#endif</pre>
	<pre>-------------------- 	
	gadget.c
	-------------------- 	
	#include "gadget.h"
	...</pre><pre>--------------------
	widget.c
	-------------------- 	
	#include "gadget.h"
	#include "thing.h"
	int main() { ... }</pre>

These source files are to be used to build an executable program file named widget, whose main function is in the source file widget.c and which uses all of the functions defined in all of the above source files.

Answer the questions below using the above information.

(a) Recall that we can specify the dependencies between files in a program using a graph, where there is an arrow drawn from each file name to the file(s) it depends on. For example, the drawing (foo→foo.o→foo.c) shows how we would diagram an executable program named foo that depends on (is built from) foo.o, which in turn is depends on foo.c. In the space below, draw a graph (diagram) showing the dependencies between the executable program widget and all of the source (.c), header (.h), and compiled (.o) files involved in building it from the files above.

Answer:



(b) Write the contents of a Makefile whose default target builds the program widget, and which only recompiles individual files as needed. Your Makefile should reflect the dependency graph you drew in part (a).

Answer:

```
widget: widget.o thing.o gadget.o
  gcc -o widget widget.o thing.o gadget.o
widget.o: widget.c gadget.h thing.h
  gcc -c widget.c
thing.o: thing.c thing.h gadget.h
  gcc -c thing.c
gadget.o: gadget.c gadget.h
  gcc -c gadget.c
```

Question 3. C++ Programming & Debugging

A student has written a program for checking whether a point is on a given straight line:

```
line 1:   #include <iostream>
line 2:   using namespace std;
line 3:   struct Point {
line 4:      int X;
line 5:      double Y;
line 6:   };
line 7:   struct Line {
line 8:      Point End1;
line 9:      Point End2;
line 10:  };
line 11:  bool Online(Point P, Line L) {
line 12:     int Slope1 =
               (L.End2.Y - L.End1.Y) / (L.End2.X - L.End1.X);
line 13:     int Slope2 = (P.Y - L.End1.Y) / (P.X - L.End1.X);
line 14:     return (Slope1 = Slope2);
line 15:  }
line 16:  int main() {
line 17:     Point P1 = {11, 12};
line 18:     Point P2 = {69, 63};
line 19:     Point P3 = {33, 29};
line 20:     Point P4 = {40, 37.5};
line 21:     Line L = {P1, P2};
line 22:     cout << Online(P3, L) << endl;
line 23:     cout << Online(P4, L) << endl;
line 24:     return 0;
line 25:  }
```

The program is expected to produce the following output:

```
0
1
```

which means that P3 = (33, 29) is not on L (a line joining P1 and P2) while P4 = (40, 37.5) is on L.

Unfortunately, his program contains some errors and the output is as follows:

```
0
0
```

Fix all the errors in the program to produce the expected output. Specify clearly the line(s) to be changed.

Answer:

Change line 12 to

```
  double Slope1 =
      (L.End2.Y - L.End1.Y) / (L.End2.X - L.End1.X);
```

Change line 13 to

```
  double Slope2 =
         (P.Y - L.End1.Y) / (P.X - L.End1.X);
```

Change line 14 to

```
  return (Slope1 == Slope2);
```

Question 4. C++ Functions & File I/O

“Selection Sort” is a technique to sort an array of elements into ascending or descending order. In this technique, the array of elements is divided into two parts. In one part all elements are sorted and in another part the items are unsorted. At first, we take the maximum or minimum data from the array. After getting the data, we place it at the beginning of the array by replacing the data of first place with the maximum or minimum data. We then consider the unsorted part and perform replacement in the same way. This process is repeated again and again until the unsorted part contains only one element.

Consider an array of integers [2, 8, 5, 7, 1, 4, 6, 9, 3] and let us use Selection Sort to sort the integers into ascending order. We first take the minimum data (i.e., 1) and replace the data of first place (i.e., 2) with it. The array becomes [1, 8, 5, 7, 2, 4, 6, 9, 3]. Next, we consider the unsorted part [8, 5, 7, 2, 4, 6, 9, 3]. We take the minimum data (i.e., 2) and replace the data of first place (i.e., 8) with it. The array becomes [1, 2, 5, 7, 8, 4, 6, 9, 3]. Next, we consider the unsorted part [5, 7, 8, 4, 6, 9, 3]. We take the minimum data (i.e., 3) and replace the data of first place (i.e., 5) with it. The array becomes [1, 2, 3, 7, 8, 4, 6, 9, 5]. We repeat this process and obtain the arrays [1, 2, 3, 4, 8, 7, 6, 9, 5], [1, 2, 3, 4, 5, 7, 6, 9, 8], [1, 2, 3, 4, 5, 6, 7, 9, 8], [1, 2, 3, 4, 5, 6, 7, 9, 8] and [1, 2, 3, 4, 5, 6, 7, 8, 9] step by step.

In this question, let us write a C++ program to read a text file named “infile.txt”, which contains a list of words (one in a line). The program then sorts the words in alphabetical order and prints the sorted list of words onto the screen.

Suppose “infile.txt” contains the following:

```
Python
Java
JavaScript
C#
C
C++
PHP
Objective-C
Swift
Kotlin
Visual Basic
Dart
Perl
```

The output of the program should be as follows:

```
C
C#
C++
Dart
Java
JavaScript
```

```
Kotlin
Objective-C
PHP
Perl
Python
Swift
Visual Basic
```

The skeleton of the program is given below.

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

void readFile(string f, string w[], int &n) {
   // To be implemented
}

void swap(string &a, string &b) {
   // To be implemented
}

void selectionSort(string w[], int n) {
   // To be implemented
}

int main() {
     string file_name = "infile.txt";
     string words[100];
     int num_words;
     readFile(file_name, words, num_words);
     selectionSort(words, num_words);
     for (int i = 0; i < num_words; i++)
          cout << words[i] << endl;
     return 0;
}
```

(a) Write down the Linux command to sort the words in "infile.txt" and print them onto the screen.

Answer:

```
sort infile.txt
```

(b) Implement the `readFile()` function with the function header
`void readFile(string f, string w[], int &n).`
This function accepts 3 parameters `f`, `w[]` and `n`. `f` is an input string parameter representing the input file name. `w[]` and `n` are output parameters for holding the words and the number of words, respectively, in the input file.

Answer:

```
void readFile(string f, string w[], int &n) {
        ifstream fin;
        fin.open(f.c_str());
        if (fin.fail()) {
                cout << "Error opening file" << endl;
                exit(1);
        }
        int count = 0;
        string line;
        while (getline(fin, line)) {
                w[count] = line;
                count ++;
        }
        fin.close();
        n = count;
}
```

(c) Implement the `swap()` function with the function header
`void swap(string &a, string &b).`
This function accepts 2 parameters a and b, both being passed by reference. The function swaps their contents.

Answer:

```
void swap(string &a, string &b) {
        string temp;
        temp = a;
        a = b;
        b = temp;
}
```

(d) Implement the function `selectionSort()` with the function header `void selectionSort(string w[], int n)`.

This function accepts 2 parameters `w[]` and `n`. `w[]` is a string array that contains a list of words while `n` is an integer storing the number of words in `w[]`. `w[]` is passed by reference by default while n is passed by value. The function then sorts the strings in `w[]` in alphabetical order.

Answer:

```
void selectionSort(string w[], int n) {
       int i, j, imin;
       for (i = 0; i < n - 1; i ++) {
              imin = i;
              for (j = i + 1; j < n; j ++)
                     if (w[j] < w[imin])
                            imin = j;
              swap(w[i], w[imin]);
       }
}
```

Question 5. C++ Linked List

You have a well-formed linked list, where the last node's next points to NULL. You do not need to perform any error checking or write any comments. Only write the requested method; do not write the surrounding class or add any instance variables. Consider this simple ListNode class.:

```
struct ListNode {
   int val;
   ListNode *next;
}
```

Add a `containsDups` function that returns true if the list contains any duplicate values, and false otherwise. For example, `containsDups(ListNode *head)` should return 1 if `head` is a `ListNode` pointer that points to the head of a linked list containing the values 3, 5, 3, 2, 0, but return 0 if it is the head of a list containing 3, 0, 2, -1.

Answer:

```
public int containsDups(ListNode *head) {

   for (ListNode *n = head; n->next != NULL; n = n->next) {
      int v = n->val;
      for (ListNode *p = n->next; p != NULL; p = p->next)
         if (p->val == v)
            return 1;
   }
   return 0;

}
```

Question 6. C++ Standard Template Library (STL)

Implement function `undup`. The input to this function is a list of strings that are not sorted in any particular order. The function should return a pointer to a newly allocated list of strings on the heap that is a copy of the original list, except that if the original list contains two or more adjacent copies of some string, those adjacent copies should be replaced by a single copy of that string. For example, if the input list contains:

```
{"apple", "donut", "donut", "banana", "banana", "banana",
"cherry", "cherry", "banana", "donut"}
```

Then `undup` should return a pointer to a new `list<string>` containing the following:

```
{"apple", "donut", "banana", "cherry", "banana", "donut"}
```

Complete the definition of function `undup` below. Some useful `#include` directives as well as a `using` directive have been provided for convenience. You should add any additional libraries or code that you need (although the sample solution only needed the ones given here).

Answer:

```
#include <string>
#include <list>
using namespace std;
// return a pointer to a new heap-allocated copy of lst
// where adjacent duplicate strings in lst are replaced
// by a single copy of that string.
list<string> * undup(const list<string> &lst) {

    list<string> * ans = new list<string>(); // result
    string prev = ""; // most recent string added to ans
    for (list<string>::iterator it = lst.begin(); it != lst.end();
   ++it) {
        if (*it != prev) {
            prev = *it;
            ans->push_back(*it);
        }
    }
    return ans;

}
```

Question 7. C array

Write the function `reverseArray` which takes an array of ints (`array`), and the number of items in that array (`n`) and reverses the contents of the array. This function should modify the original array.

Answer:

```
void reverseArray (int * array, int n) {


  for (int i = 0; i < n/2; i++) {
    int temp = array[i];
    array[i] = array[n-i-1];
    array[n-i-1] = temp;
  }

}
```

== End of sample exam paper! ☺ ==